OPENVINO ADVANCED · VISION PROJECTS

OpenVINO 심화와 비전 AI 프로젝트

추론 파이프라인 · 디바이스 최적화 · 물류 계수 · 이상 탐지 품질검사

OpenVINO의 모델 로딩부터 디바이스 최적화까지 깊이 이해하고, 이를 바탕으로 물류 자동 계수 시스템과 비지도 이상 탐지 품질검사 시스템을 직접 구축합니다.

CONTENTS

목차

1 **OpenVINO 추론 기초** 03 – 98

모델 로딩/컴파일·전후처리, 분류·검출·자세추정+웹캠, 디바이스 비교·AUTO·INT8

2 **사전학습 모델 활용** 99 – 154

이미지 분류·객체 검출·자세 추정 모델 실행, 웹캠 실시간 연동

3 **최적화와 디바이스** 155 – 198

CPU·GPU·NPU 성능 비교, AUTO 모드, INT8 양자화와 정확도·속도 분석

4 **실습과제** 99 – 154

실시간 객체 검출·계수 시스템 (물류 자동 계수 사례 재현)

PART

1

OpenVINO 추론 기초

모델을 읽고 컴파일하는 방법부터 전/후처리 구조, 분류·검출·자세추정 모델 실행과 웹캠 연동, 그리고 CPU/GPU/NPU 성능 비교와 AUTO 모드·INT8 양자화까지 OpenVINO 추론의 기초를 깊이 있게 다룹니다.

OpenVINO 추론 파이프라인이란?

OpenVINO로 모델을 실행하는 과정은 항상 같은 4단계를 거칩니다. 이 구조를 한 번 제대로 이해하면, 어떤 모델을 다루든 같은 틀로 접근할 수 있습니다.

- **모델 읽기 (read_model)**

  IR/ONNX 파일을 디바이스와 무관한 ov.Model 객체로 불러옵니다.

- **컴파일 (compile_model)**

  특정 디바이스(CPU/GPU/NPU)에 맞춰 실행 가능한 형태로 변환합니다.

- **추론 (infer)**

  전처리된 입력을 넣고 결과 텐서를 얻습니다.

- **후처리 (postprocess)**

  raw 출력 텐서를 사람이 이해할 수 있는 결과(클래스, 박스 등)로 바꿉니다.

파이프라인 4단계 한눈에

단계	핵심 API	비고
모델 읽기	core.read_model(path)	디바이스 독립적인 ov.Model 반환
컴파일	core.compile_model(model, device)	디바이스별 최적화, 비용이 큰 단계
추론	compiled_model(inputs) 또는 InferRequest	결과 텐서(raw) 획득
후처리	argmax, NMS, 박스 디코딩 등	모델 종류에 따라 방식이 다름

Core 객체와 read_model

코드

load_model.py

```
import openvino as ov

core = ov.Core()

# IR(.xml/.bin), ONNX 등을 읽어 device-독립적인 ov.Model로 반환
model = core.read_model(model="model.xml")
print(model.inputs)          # 입력 텐서 정보
print(model.outputs)         # 출력 텐서 정보
```

NOTE read_model 단계에서는 아직 어떤 디바이스에서도 실행할 수 없습니다. 컴파일 전까지는 순수한 그래프 구조일 뿐입니다.

read_model vs compile_model

개념

read_model

- 파일을 파싱해 ov.Model 객체를 만듭니다.
- 디바이스와 무관합니다 — 아직 실행 불가.
- 비교적 빠르게 끝납니다.

compile_model

- ov.Model을 특정 디바이스에 맞게 최적화·컴파일합니다.
- CompiledModel을 반환하며, 이걸로 추론을 실행합니다.
- 디바이스·모델 크기에 따라 시간이 걸릴 수 있습니다.

compile_model로 디바이스 지정하기

코드

compile_model.py

```
compiled_model = core.compile_model(model=model, device_name="CPU")

# 읽기+컴파일을 한 번에 하는 축약형도 있습니다
compiled_model = ov.compile_model("model.xml", "GPU")
```

NOTE compile_model에는 ov.Model 객체 대신 파일 경로를 바로 넘길 수도 있습니다.

IR 모델 파일 구성

코드

항목	설명
`.xml`	모델의 네트워크 구조(레이어, 연결 관계)를 담은 파일입니다.
`.bin`	학습된 가중치(weight)와 편향(bias) 값을 담은 바이너리 파일입니다.
`.mapping`	원본 프레임워크의 텐서 이름과 IR 텐서 이름을 연결하는 매핑 파일입니다.

추론을 실행하는 두 가지 방법

개념

Callable 방식

- compiled_model(inputs)처럼 함수처럼 바로 호출합니다.
- 코드가 짧고 간단한 스크립트에 적합합니다.
- 반복 호출이 많은 경우 약간의 오버헤드가 있습니다.

InferRequest 방식

- create_infer_request()로 요청 객체를 만들어 재사용합니다.
- 반복 추론(웹캠, 배치)에 더 효율적입니다.
- 비동기 추론(start_async)으로 확장하기 쉽습니다.

Callable 방식으로 추론하기

코드

infer_callable.py

```
output_layer = compiled_model.output(0)

result = compiled_model(input_data)[output_layer]
print(result.shape)
```

InferRequest 방식으로 추론하기

코드

infer_request.py

```
infer_request = compiled_model.create_infer_request()

infer_request.infer(inputs={0: input_data})

output = infer_request.get_output_tensor(0).data
print(output.shape)
```

Core 객체의 주요 메서드

코드

항목	설명
`core.read_model()`	모델 파일을 ov.Model로 읽어옵니다.
`core.compile_model()`	모델을 특정 디바이스용으로 컴파일합니다.
`core.available_devices`	현재 시스템에서 사용 가능한 디바이스 목록을 반환합니다.
`core.get_property()`	디바이스나 컴파일된 모델의 속성 값을 조회합니다.
`core.set_property()`	캐시 경로 등 전역 설정 값을 지정합니다.

컴파일 결과 캐싱하기

코드

model_cache.py

```
core.set_property({"CACHE_DIR": "./model_cache"})

compiled_model = core.compile_model(model=model, device_name="GPU")

# 다음 실행부터는 캐시를 재사용해 컴파일 시간이 크게 줄어듭니다
```

NOTE 특히 GPU·NPU는 컴파일(커널 최적화)에 시간이 걸리므로, 캐싱을 켜두면 애플리케이션 시작 속도가 크게 개선됩니다.

캐싱이 필요한 이유

개념

캐싱 없이

- 앱을 실행할 때마다 매번 컴파일을 새로 합니다.
- GPU/NPU는 이 과정이 수 초씩 걸릴 수 있습니다.
- 웹캠 앱처럼 빠른 시작이 중요하면 불리합니다.

캐싱 사용 시

- 최초 1회만 컴파일하고 결과를 디스크에 저장합니다.
- 이후 실행은 캐시를 불러오기만 하면 됩니다.
- 모델·디바이스 조합이 바뀌면 캐시도 새로 생성됩니다.

직접 해보기

실습

TASK

분류 모델 하나를 read_model로 불러오고, CPU로 compile_model 한 뒤, CACHE_DIR을 설정해 두 번째 실행부터 속도가 어떻게 달라지는지 시간을 측정해보세요.

TIP

time.time()으로 compile_model 앞뒤 시간을 측정하면 캐싱 효과를 눈으로 바로 확인할 수 있습니다.

입력 전처리, 왜 필요한가

모델은 학습될 때 정해진 형태의 입력만 받아들입니다. 우리가 가진 원본 이미지는 크기도, 채널 순서도, 값의 범위도 제각각이라서 반드시 맞춰줘야 합니다.

크기 불일치

카메라 이미지는 보통 모델이 원하는 224x224, 640x640 등과 다릅니다.

채널 순서

OpenCV는 BGR, 대부분의 모델은 RGB를 기준으로 학습됩니다.

값의 범위

픽셀 값 0~255를 모델이 원하는 0~1 또는 정규화된 값으로 바꿔야 합니다.

차원 순서

HWC(높이,너비,채널) 이미지를 모델이 원하는 CHW로 바꿔야 할 때가 많습니다.

NCHW vs NHWC 레이아웃

개념

NHWC

- 배치, 높이, 너비, 채널 순서입니다.
- OpenCV로 읽은 이미지의 기본 형태와 같습니다.
- TensorFlow 계열 모델에서 흔히 사용됩니다.

NCHW

- 배치, 채널, 높이, 너비 순서입니다.
- PyTorch 계열 모델(YOLO 포함)의 기본 형태입니다.
- OpenVINO의 많은 모델이 이 레이아웃을 기대합니다.

OpenCV/NumPy로 직접 전처리하기

코드

manual_preprocess.py

```
image = cv2.imread("photo.jpg")
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)    # BGR -> RGB
resized = cv2.resize(image, (640, 640))             # 모델 입력 크기로

chw = resized.transpose(2, 0, 1)                    # HWC -> CHW
input_tensor = np.expand_dims(chw, 0) / 255.0       # 배치 차원 + 0~1 정규화
```

수동 전처리의 한계

개념

수동 전처리 (NumPy/OpenCV)

- 코드가 직관적이고 이해하기 쉽습니다.
- CPU에서 실행되어 디바이스 자원을 활용하지 못합니다.
- 전처리량이 많으면 병목이 될 수 있습니다.

PrePostProcessor (PPP)

- 전처리 단계를 모델 그래프 안에 포함시킵니다.
- 실제 추론 디바이스에서 함께 실행되어 더 빠릅니다.
- 코드는 조금 더 길지만 한 번 설정하면 재사용이 쉽습니다.

PrePostProcessor 기본 구조

코드

ppp_basic.py

```
import openvino as ov

ppp = ov.preprocess.PrePostProcessor(model)

# 실제로 우리가 넣을 입력의 형태를 그대로 설명
ppp.input().tensor() \
    .set_shape(input_data.shape) \
    .set_element_type(ov.Type.u8) \
    .set_layout(ov.Layout("NHWC"))
```

PPP로 리사이즈·레이아웃 변환 넣기

코드

ppp_resize.py

```
ppp.input().preprocess() \
    .resize(ov.preprocess.ResizeAlgorithm.RESIZE_LINEAR)

# 모델이 실제로 기대하는 레이아웃을 지정
ppp.input().model().set_layout(ov.Layout("NCHW"))

model = ppp.build()
compiled_model = core.compile_model(model, "CPU")
```

PPP로 정규화(mean/scale) 넣기

코드

ppp_normalize.py

```
ppp.input().preprocess() \
    .mean([123.675, 116.28, 103.53]) \
    .scale([58.395, 57.12, 57.375])

ppp.output().tensor().set_element_type(ov.Type.f32)

model = ppp.build()
```

NOTE mean/scale 값은 모델이 학습될 때 사용된 정규화 값과 반드시 일치해야 합니다.

PrePostProcessor 주요 메서드

코드

항목	설명
`.input().tensor()`	실제로 넣을 입력 데이터의 shape·타입·레이아웃을 설명합니다.
`.input().preprocess()`	resize, mean/scale 등 전처리 연산을 그래프에 추가합니다.
`.input().model()`	모델이 원래 기대하는 레이아웃을 지정합니다.
`.output().tensor()`	출력 텐서의 타입 등을 지정합니다.
`.build()`	설정을 반영한 새로운 ov.Model을 만듭니다.

직접 해보기

실습

TASK

같은 이미지를 수동 전처리와 PrePostProcessor 두 가지 방식으로 각각 준비해 같은 모델에 넣어보고, 결과가 동일한지 비교해보세요.

TIP

결과가 다르다면 resize 보간 방식이나 mean/scale 값이 서로 어긋났을 가능성이 큽니다.

출력 후처리 구조

모델의 raw 출력은 숫자 텐서일 뿐입니다. 이를 사람이 이해할 수 있는 클래스명, 확률, 바운딩 박스로 바꾸는 과정이 후처리이며, 모델 종류마다 방식이 다릅니다.

- **분류 모델**

  클래스별 확률 벡터에서 가장 높은 값(argmax)을 찾습니다.

- **검출 모델 (SSD계열)**

  [박스 좌표, 클래스, 신뢰도]로 구성된 표 형태 출력을 해석합니다.

- **검출 모델 (YOLO계열)**

  앵커 기반 출력을 디코딩하고 NMS로 중복 박스를 제거합니다.

- **자세 추정 모델**

  관절(keypoint)별 좌표와 신뢰도를 추출합니다.

분류 모델 결과 해석하기

코드

classify_postprocess.py

```
output = compiled_model(input_tensor)[output_layer]
probs = output.reshape(-1)

top5 = np.argsort(probs)[-5:][::-1]      # 확률 상위 5개 인덱스

for idx in top5:
    print(f"{class_names[idx]}: {probs[idx]:.3f}")
```

SSD 계열 검출 출력 포맷

위치	의미	비고
detection[1]	클래스 ID	검출된 객체의 클래스 번호
detection[2]	신뢰도(confidence)	보통 0.5 이상만 사용
detection[3:5]	좌상단 좌표 (x_min, y_min)	0~1로 정규화된 값
detection[5:7]	우하단 좌표 (x_max, y_max)	이미지 크기를 곱해 픽셀 좌표로 변환

SSD 출력 디코딩하기

코드

detect_postprocess.py

```
detections = output.reshape(-1, 7)
h, w = image.shape[:2]

for det in detections:
    confidence = det[2]
    if confidence > 0.5:
        xmin, ymin = int(det[3]*w), int(det[4]*h)
        xmax, ymax = int(det[5]*w), int(det[6]*h)
```

검출 결과 박스 그리기

코드

draw_boxes.py

```
cv2.rectangle(image, (xmin, ymin), (xmax, ymax), (0, 255, 0), 2)

label = f"{class_names[int(det[1])]}: {confidence:.2f}"
cv2.putText(image, label, (xmin, ymin - 8),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
```

YOLO 계열 출력은 조금 다릅니다

개념

SSD 계열

- 이미 디코딩된 [박스, 클래스, 신뢰도] 표를 바로 줍니다.
- 후처리가 비교적 단순합니다.

YOLO 계열

- 앵커 기반 raw 출력을 직접 디코딩하고 NMS까지 해야 합니다.
- Ultralytics로 내보낸 모델은 자체 Results 객체로 이 과정을 대신 처리해줍니다.
- 직접 구현하기보다 Ultralytics의 내장 후처리를 활용하는 것이 안전합니다.

직접 해보기

실습

TASK

검출 모델의 출력을 디코딩해 신뢰도 0.6 이상인 객체만 이미지에 박스와 라벨로 표시해보세요.

TIP

신뢰도 기준을 너무 낮게 잡으면 오검출이 늘고, 너무 높게 잡으면 실제 객체를 놓칠 수 있습니다. 0.5~0.7 사이에서 조절해보세요.

자주 만나는 로딩·추론 오류

코드

항목	설명
파일 경로 오류	.xml 파일 경로에 .bin 파일이 같은 폴더에 없으면 로딩에 실패합니다.
`shape mismatch`	입력 텐서의 shape이 모델이 기대하는 shape과 다르면 오류가 납니다.
`device not found`	GPU/NPU 드라이버가 설치되지 않았는데 해당 디바이스를 지정하면 실패합니다.
레이아웃 불일치	NCHW/NHWC를 혼동하면 학습 시와 전혀 다른 결과가 나올 수 있습니다.

전체 파이프라인 한눈에

읽기 → 전처리 → 컴파일 → 추론 → 후처리. 이 다섯 단계는 분류든 검출이든 자세추정이든 항상 동일하게 반복됩니다. 다음 장에서는 이 구조를 실제 세 가지 태스크에 적용해봅니다.

모델 읽기·컴파일

read_model + compile_model, 필요하면 CACHE_DIR로 재사용.

전처리

수동(OpenCV/NumPy) 또는 PrePostProcessor로 모델이 원하는 형태로 변환.

추론

callable 또는 InferRequest 방식으로 결과 텐서를 얻음.

후처리

모델 종류(분류/검출/자세추정)에 맞는 방식으로 결과를 해석.

핵심 API 총정리

요약

항목	설명
`core.read_model / compile_model`	모델을 읽고 디바이스에 맞게 컴파일합니다.
`compiled_model(x) / InferRequest`	추론을 실행하는 두 가지 방식입니다.
`CACHE_DIR`	컴파일 결과를 캐싱해 재시작 속도를 높입니다.
`PrePostProcessor`	전처리를 모델 그래프에 포함시켜 디바이스에서 실행합니다.
`argmax / SSD 디코딩 / NMS`	모델 종류별 출력 후처리 방식입니다.

세 가지 비전 태스크를 직접 돌려봅니다

분류(Classification), 검출(Detection), 자세 추정(Pose Estimation)은 컴퓨터 비전에서 가장 널리 쓰이는 세 가지 태스크입니다. 이번 장에서는 Ultralytics YOLO26 모델을 OpenVINO IR로 내보내 세 태스크를 모두 실행해봅니다.

- **분류**

  이미지 전체가 어떤 클래스에 속하는지 판단.

- **검출**

  이미지 안 객체들의 위치(박스)와 클래스를 함께 찾음.

- **자세 추정**

  사람의 관절 위치(keypoint)를 찾아 자세를 추정.

Open Model Zoo 대신 Ultralytics를 사용합니다

개념

Open Model Zoo

- 과거 OpenVINO의 사전학습 모델 저장소였습니다.
- 현재는 유지보수 모드로 전환되어 새 모델이 추가되지 않습니다.
- 기존 예제 코드에 남아있지만 신규 프로젝트에는 권장되지 않습니다.

Ultralytics YOLO26

- 2026년 1월 발표된 최신 플래그십 모델로 분류·검출·자세추정을 모두 지원합니다.
- `model.export(format="openvino")` 한 줄로 OpenVINO IR 변환이 가능합니다.
- 이 강의에서는 YOLO26을 기준으로 실습합니다.

분류 모델을 OpenVINO IR로 내보내기

코드

export_classify.py

```python
from ultralytics import YOLO

model = YOLO("yolo26n-cls.pt")
model.export(format="openvino", half=True)
# 실행 후 yolo26n-cls_openvino_model/ 폴더가 생성됩니다
```

NOTE half=True는 FP16으로 내보내 파일 크기와 속도를 함께 개선합니다.

분류 모델 추론 실행하기

코드

run_classify.py

```python
from ultralytics import YOLO

ov_model = YOLO("yolo26n-cls_openvino_model/")
results = ov_model("sample.jpg", device="cpu")

for r in results:
    probs = r.probs
    print(probs.top5, probs.top5conf)
```

분류 결과에서 확인할 값들

필드	의미
probs.top1	가장 확률이 높은 클래스 인덱스
probs.top1conf	1위 클래스의 신뢰도(확률)
probs.top5	상위 5개 클래스 인덱스 리스트
probs.top5conf	상위 5개 클래스 신뢰도 리스트

직접 해보기

실습

TASK

분류 모델로 3장의 다른 이미지를 추론하고, 각 이미지의 top-1 클래스와 신뢰도를 표로 정리해보세요.

TIP

신뢰도가 낮게 나온다면 학습 데이터셋(ImageNet 등)에 없는 클래스일 가능성이 높습니다.

객체 검출 모델 실행하기

검출 모델은 이미지 안에서 여러 객체의 위치와 종류를 동시에 찾아냅니다. 절차는 분류와 거의 동일하지만, 결과에서 박스 좌표를 함께 다루게 됩니다.

입력

한 장의 이미지 또는 영상 프레임.

출력

박스 좌표(xyxy), 클래스, 신뢰도의 리스트.

후처리

Ultralytics 내부에서 NMS까지 자동으로 처리됩니다.

검출 모델을 OpenVINO IR로 내보내기

코드

export_detect.py

```
from ultralytics import YOLO

model = YOLO("yolo26n.pt")
model.export(format="openvino", half=True, imgsz=640)
# yolo26n_openvino_model/ 폴더가 생성됩니다
```

검출 모델 추론 실행하기

코드

run_detect.py

```
from ultralytics import YOLO

ov_model = YOLO("yolo26n_openvino_model/")
results = ov_model("street.jpg", device="cpu", conf=0.5)

for r in results:
    for box in r.boxes:
        print(box.xyxy, box.cls, box.conf)
```

검출 결과를 이미지에 그리기

코드

draw_detect.py

```
import cv2

result = ov_model("street.jpg")[0]
plotted = result.plot()    # 박스·라벨이 그려진 이미지(numpy) 반환
cv2.imwrite("street_result.jpg", plotted)
```

NOTE result.plot()은 Ultralytics가 제공하는 기본 시각화 함수로, 직접 cv2.rectangle을 쓰지 않아도 됩니다.

검출 결과 boxes 객체의 주요 필드

코드

항목	설명
`box.xyxy`	(x1, y1, x2, y2) 형태의 박스 좌표(픽셀 단위).
`box.xywhn`	중심점·너비·높이를 0~1로 정규화한 좌표.
`box.cls`	검출된 객체의 클래스 인덱스.
`box.conf`	검출 신뢰도(0~1).

직접 해보기

실습

TASK

검출 모델로 사람이 포함된 이미지를 추론하고, 신뢰도 0.5 이상인 사람 객체 수를 세어 출력해보세요.

TIP

box.cls 값과 model.names 딕셔너리를 비교하면 클래스 이름을 확인할 수 있습니다.

자세 추정 모델 실행하기

자세 추정 모델은 사람의 몸에서 17개의 주요 관절(keypoint) 위치를 찾아냅니다. 운동 자세 분석, 낙상 감지, 동작 인식 등에 활용됩니다.

- **keypoint**

  코, 눈, 어깨, 팔꿈치, 손목, 엉덩이, 무릎, 발목 등 17개 지점.

- **출력 형태**

  사람마다 (x, y, 신뢰도) × 17개 배열.

- **활용**

  스켈레톤을 그려 자세를 시각적으로 확인.

자세 추정 모델을 OpenVINO IR로 내보내기

코드

export_pose.py

```
from ultralytics import YOLO

model = YOLO("yolo26n-pose.pt")
model.export(format="openvino", half=True)
# yolo26n-pose_openvino_model/ 폴더가 생성됩니다
```

자세 추정 추론과 스켈레톤 그리기

코드

run_pose.py

```
from ultralytics import YOLO

ov_model = YOLO("yolo26n-pose_openvino_model/")
result = ov_model("person.jpg")[0]
plotted = result.plot()    # 관절과 스켈레톤이 그려진 이미지

keypoints = result.keypoints.xy    # (사람 수, 17, 2)
```

사람 자세 keypoint 17개 인덱스 (COCO 기준)

인덱스	부위	인덱스	부위
0	코	9	왼쪽 손목
1~2	눈(좌/우)	10	오른쪽 손목
3~4	귀(좌/우)	11~12	엉덩이(좌/우)
5~6	어깨(좌/우)	13~14	무릎(좌/우)
7~8	팔꿈치(좌/우)	15~16	발목(좌/우)

직접 해보기

실습

TASK

자세 추정 모델로 사람 이미지를 추론하고, 왼쪽 손목(9번)과 오른쪽 손목(10번) keypoint의 좌표를 출력해보세요.

TIP

신뢰도가 낮은 keypoint(가려진 부위 등)는 좌표가 부정확할 수 있으니 신뢰도 값도 함께 확인하세요.

2-1~2-3 · 비교

세 모델, 무엇이 다른가

구분	분류	검출	자세추정
출력	클래스 확률	박스+클래스+신뢰도	keypoint 좌표
모델 예시	yolo26n-cls	yolo26n	yolo26n-pose
대표 활용	이미지 태깅	물류 계수, 보안	동작 인식, 낙상 감지

웹캠으로 실시간 추론하기

정지 이미지 추론에 익숙해졌다면, 이제 웹캠 프레임을 실시간으로 받아 추론하는 방법을 알아봅니다. 핵심은 OpenCV로 프레임을 캡처하고, 매 프레임마다 추론을 반복하는 것입니다.

캡처

cv2.VideoCapture(0)으로 웹캠 스트림을 엽니다.

루프

프레임을 하나씩 읽어 추론하고 화면에 표시합니다.

종료

키 입력을 받아 루프를 빠져나오고 자원을 해제합니다.

OpenCV로 웹캠 열기

코드

webcam_capture.py

```
import cv2

cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise RuntimeError("웹캠을 열 수 없습니다")

ret, frame = cap.read()
print(frame.shape)    # (높이, 너비, 채널)
```

프레임별 추론 루프 만들기

코드

webcam_infer_loop.py

```
while True:
    ret, frame = cap.read()
    if not ret:
        break

    result = ov_model(frame, verbose=False)[0]
    plotted = result.plot()
    cv2.imshow("OpenVINO Webcam", plotted)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
```

FPS 측정하기

코드

fps_measure.py

```
import time

prev = time.time()
now = time.time()
fps = 1.0 / (now - prev)
prev = now
cv2.putText(plotted, f"FPS: {fps:.1f}", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
```

NOTE 매 프레임 시각 차이의 역수가 순간 FPS입니다. 여러 프레임의 평균을 내면 더 안정적입니다.

실시간 검출 데모 완성본

코드

webcam_detect_demo.py

```
from ultralytics import YOLO
import cv2, time

model = YOLO("yolo26n_openvino_model/")
cap = cv2.VideoCapture(0)
prev = time.time()

while True:
    ret, frame = cap.read()
    if not ret: break
    result = model(frame, device="cpu", verbose=False)[0]
    plotted = result.plot()
    now = time.time(); fps = 1/(now-prev); prev = now
```

NOTE 분류·자세추정 데모도 model만 바꿔주면 동일한 구조로 동작합니다.

웹캠 자원 정리하기

코드

webcam_cleanup.py

```
cap.release()
cv2.destroyAllWindows()
# 웹캠 핸들을 반드시 해제해야 다른 프로그램이 다시 사용할 수 있습니다
```

이미지 · 동영상 · 웹캠, 코드는 얼마나 다를까

개념

이미지 / 동영상 파일

- `ov_model("file.jpg")` 또는 `ov_model("file.mp4")`처럼 경로만 넘기면 됩니다.
- 동영상은 내부적으로 프레임을 순회하며 자동 처리됩니다.

웹캠

- `cv2.VideoCapture(0)`으로 직접 프레임을 받아야 합니다.
- 루프·종료 조건·자원 해제를 직접 관리해야 합니다.

웹캠 연동에서 자주 겪는 문제

코드

항목	설명
`cap.isOpened() == False`	웹캠 장치 번호(0, 1...)가 다르거나 다른 프로그램이 점유 중입니다.
화면이 멈춘 것처럼 보임	waitKey(1)이 없으면 imshow 창이 갱신되지 않습니다.
FPS가 너무 낮음	해상도를 낮추거나 device="gpu"/"npu"로 바꿔보세요.
창이 안 닫힘	destroyAllWindows()를 빠뜨리면 창이 남을 수 있습니다.

실시간 추론을 더 매끄럽게 만드는 방법

1 입력 해상도 축소

640보다 작은 imgsz로 내보내면 속도가 크게 향상됩니다.

2 FP16/INT8 활용

half=True 또는 양자화된 모델을 쓰면 지연시간이 줄어듭니다.

3 device 전환

CPU 대신 GPU/NPU/AUTO를 지정해 처리량을 높일 수 있습니다.

4 불필요한 출력 끄기

verbose=False로 콘솔 로그 오버헤드를 줄입니다.

동기 추론 vs 비동기 추론

개념

동기(Sync)

- 추론이 끝날 때까지 다음 코드가 대기합니다.
- 구현이 단순하지만 CPU가 노는 시간이 생길 수 있습니다.
- 이번 장의 예제는 모두 동기 방식입니다.

비동기(Async)

- `InferRequest`의 `start_async()`로 추론을 백그라운드에서 실행합니다.
- 다음 프레임을 준비하는 동안 이전 프레임 추론을 겹쳐 처리할 수 있습니다.
- 실시간 스트리밍에서 처리량을 끌어올릴 때 사용합니다.

직접 해보기

실습

TASK

웹캠 검출 데모에 FPS 표시를 추가하고, device를 "cpu"에서 다른 값으로 바꿔가며 체감 속도 차이를 비교해보세요.

TIP

같은 조건에서 비교하려면 imgsz와 conf 값을 고정한 채 device만 바꿔야 합니다.

핵심 API 총정리

요약

항목	설명
`model.export(format="openvino")`	Ultralytics 모델을 OpenVINO IR로 변환합니다.
`YOLO("...openvino_model/")`	변환된 IR 모델을 다시 불러와 추론에 사용합니다.
`result.plot()`	검출·분류·자세추정 결과를 이미지에 시각화합니다.
`cv2.VideoCapture(0)`	웹캠 스트림을 열어 프레임을 순차적으로 읽습니다.

어떤 디바이스에서 실행할까

OpenVINO는 같은 모델을 CPU, GPU, NPU 등 여러 하드웨어에서 실행할 수 있게 해줍니다. 디바이스마다 성능·전력·정확도 특성이 다르므로, 상황에 맞는 선택이 중요합니다.

- **CPU**

  항상 사용 가능하며 범용적이지만 상대적으로 느립니다.

- **GPU(iGPU)**

  병렬 연산에 강해 배치 처리·고해상도에 유리합니다.

- **NPU**

  저전력 상시 추론에 최적화된 전용 가속기입니다.

사용 가능한 디바이스 조회하기

코드

available_devices.py

```
import openvino as ov

core = ov.Core()
print(core.available_devices)
# 예: ['CPU', 'GPU', 'NPU']

print(core.get_property("CPU", "FULL_DEVICE_NAME"))
```

benchmark_app 옵션 다시 보기

코드

항목	설명
`-d CPU / GPU / NPU`	벤치마크를 실행할 디바이스를 지정합니다.
`-hint throughput / latency`	처리량 우선 또는 지연시간 우선 모드를 선택합니다.
`-niter`	반복 추론 횟수를 지정합니다.
`-api sync / async`	동기/비동기 실행 방식을 선택합니다.

CPU에서 벤치마크 실행하기

코드

benchmark_cpu.py

```
import subprocess

subprocess.run([
    "benchmark_app", "-m", "model.xml",
    "-d", "CPU", "-hint", "throughput"
])
```

GPU에서 벤치마크 실행하기

코드

benchmark_gpu.py

```
subprocess.run([
    "benchmark_app", "-m", "model.xml",
    "-d", "GPU", "-hint", "throughput"
])
# iGPU가 없으면 device not found 오류가 발생합니다
```

NPU에서 벤치마크 실행하기

코드

benchmark_npu.py

```
subprocess.run([
    "benchmark_app", "-m", "model.xml",
    "-d", "NPU", "-hint", "latency"
])
# NPU는 낮은 지연시간·저전력 상시 추론에 강점이 있습니다
```

처리량(Throughput) vs 지연시간(Latency)

개념

처리량 (Throughput)

- 초당 처리 가능한 프레임 수(FPS).
- 여러 스트림을 동시에 처리할 때 중요합니다.
- -hint throughput으로 최적화합니다.

지연시간 (Latency)

- 한 번의 추론이 끝나는 데 걸리는 시간(ms).
- 실시간 반응성이 중요한 애플리케이션에서 중요합니다.
- -hint latency로 최적화합니다.

직접 해보기

실습

TASK

같은 모델을 CPU와 GPU(또는 NPU)에서 각각 benchmark_app으로 실행하고 처리량(FPS)을 비교해보세요.

TIP

디바이스가 하나뿐이라면 -hint를 throughput과 latency로 바꿔가며 비교해도 좋습니다.

AUTO 모드, 디바이스 선택을 맡기기

매번 디바이스를 직접 지정하는 대신, OpenVINO의 AUTO 플러그인에게 최적의 디바이스 선택을 맡길 수 있습니다. AUTO는 사용 가능한 디바이스 중 가장 적합한 것을 자동으로 고릅니다.

동작

모델과 힌트를 보고 가장 성능이 좋을 것으로 예측되는 디바이스를 선택합니다.

장점

하드웨어 구성이 다른 여러 환경에 배포할 때 코드를 바꾸지 않아도 됩니다.

AUTO 디바이스로 컴파일하기

코드

auto_mode.py

```
compiled_model = core.compile_model(model, "AUTO")
print(compiled_model.get_property("EXECUTION_DEVICES"))
# 실제로 선택된 디바이스 이름이 출력됩니다
```

AUTO에 우선순위 리스트 지정하기

코드

auto_priority.py

```
compiled_model = core.compile_model(model, "AUTO:GPU,CPU")
# GPU를 우선 시도하고, 불가능하면 CPU로 전환(fallback)합니다
```

AUTO 모드의 내부 동작

개념

초기 컴파일

- 첫 요청은 컴파일이 빠른 CPU로 즉시 처리해 초기 지연을 줄입니다.
- 백그라운드에서 목표 디바이스(GPU 등) 컴파일을 진행합니다.

전환

- 목표 디바이스 컴파일이 끝나면 이후 추론은 그 디바이스로 넘어갑니다.
- 사용자 코드는 이 전환을 신경 쓸 필요가 없습니다.

성능 힌트로 목적에 맞게 조정하기

개념

performance_mode

- THROUGHPUT: 처리량을 최대화합니다.
- LATENCY: 지연시간을 최소화합니다.
- 기본값은 대체로 LATENCY입니다.

적용 방법

- `ov.properties.hint.performance_mode` 프로퍼티로 설정합니다.
- compile_model 호출 시 config 딕셔너리로 전달합니다.

성능 힌트 설정 코드

코드

perf_hint.py

```
import openvino.properties.hint as hints

compiled_model = core.compile_model(
    model, "AUTO",
    {hints.performance_mode: hints.PerformanceMode.THROUGHPUT}
)
```

직접 해보기

실습

TASK

"AUTO:GPU,CPU"와 "AUTO:NPU,CPU" 두 가지 우선순위로 각각 컴파일하고, EXECUTION_DEVICES 값을 비교해보세요.

TIP

해당 디바이스가 시스템에 없으면 자동으로 다음 순위 디바이스가 선택되는지 확인해보세요.

INT8 양자화란

양자화(Quantization)는 모델의 가중치·연산을 FP32/FP16보다 낮은 정밀도(INT8)로 표현해 속도와 메모리 효율을 높이는 기법입니다. 정확도 손실을 최소화하면서 추론을 가속합니다.

목표

정확도 손실을 최소화하면서 속도·메모리 효율을 높이는 것.

대상

특히 CPU와 NPU에서 INT8 연산 가속 효과가 큽니다.

FP32 · FP16 · INT8 비교

항목	FP32	FP16	INT8
정확도	기준(가장 높음)	거의 동일	약간 손실 가능
속도	가장 느림	더 빠름	가장 빠름
모델 크기	가장 큼	약 1/2	약 1/4
주 사용처	학습·정밀 검증	일반 추론	엣지·실시간 추론

NNCF로 양자화합니다 (POT는 폐지됨)

개념

과거: POT

- Post-training Optimization Tool, 과거 OpenVINO의 양자화 도구였습니다.
- 현재는 완전히 폐지(deprecated → removed)되어 더 이상 사용되지 않습니다.

현재: NNCF

- Neural Network Compression Framework, OpenVINO 공식 양자화 도구입니다.
- Post-training 양자화와 학습 중 양자화(QAT)를 모두 지원합니다.
- 이 강의에서는 NNCF의 post-training 양자화를 사용합니다.

NNCF 설치하기

코드

install_nncf.sh

```
pip install nncf

# 설치 확인
python -c "import nncf; print(nncf.__version__)"
```

NOTE OpenVINO와 NNCF는 버전 호환성이 중요하므로, 사용 중인 OpenVINO 버전에 맞는 NNCF를 설치하는 것이 좋습니다.

양자화용 보정(Calibration) 데이터셋 만들기

코드

calibration_dataset.py

```python
import nncf

def transform_fn(data_item):
    image, _ = data_item
    return preprocess(image)    # 모델 입력 형태로 전처리

calibration_dataset = nncf.Dataset(val_loader, transform_fn)
```

NOTE 보정 데이터는 실제 추론 데이터와 분포가 비슷한 300장 내외면 충분합니다.

nncf.quantize로 INT8 모델 만들기

코드

quantize_model.py

```
quantized_model = nncf.quantize(
    model,
    calibration_dataset,
    subset_size=300
)
```

양자화된 모델 저장하고 실행하기

코드

save_quantized.py

```
import openvino as ov


ov.save_model(quantized_model, "model_int8.xml")


core = ov.Core()
compiled = core.compile_model("model_int8.xml", "CPU")
```

정확도와 속도, 균형을 맞추기

개념

속도 향상

- CPU에서 2~4배가량 처리량이 개선되는 경우가 흔합니다.
- NPU에서는 INT8이 기본적으로 최적화된 경로입니다.

정확도 확인

- 양자화 후 검증셋으로 mAP/accuracy를 재측정해야 합니다.
- 손실이 크면 보정 데이터 양을 늘리거나 QAT를 고려합니다.

직접 해보기

실습

TASK

FP16 모델과 INT8 양자화 모델을 각각 benchmark_app으로 측정해 처리량 차이를 비교해보세요.

TIP

같은 -hint, 같은 -niter 조건에서 비교해야 공정한 비교가 됩니다.

양자화에서 자주 하는 실수

코드

항목	설명
보정 데이터 부족	너무 적은 샘플로 보정하면 정확도 손실이 커질 수 있습니다.
분포가 다른 보정 데이터	실제 서비스 데이터와 다른 분포면 양자화 효과가 왜곡됩니다.
검증 생략	양자화 후 정확도 재측정을 생략하면 품질 저하를 놓칠 수 있습니다.
POT 예제 코드 참고	오래된 POT 기반 예제는 더 이상 동작하지 않으니 NNCF 문서를 참고하세요.

상황별 디바이스 선택 가이드

1 개발·디버깅

항상 사용 가능하고 예측 가능한 CPU를 우선 사용합니다.

2 고해상도·배치 처리

병렬 연산에 강한 GPU가 유리한 경우가 많습니다.

3 저전력 상시 추론

엣지 기기에서는 NPU가 전력 대비 효율이 좋습니다.

4 배포 환경이 다양할 때

AUTO 모드로 하드웨어 차이를 코드 변경 없이 흡수합니다.

양자화를 적용하기 좋은 상황

1 엣지 디바이스 배포

메모리·연산 자원이 제한된 환경일수록 효과가 큽니다.

2 실시간 처리 요구

지연시간을 줄여야 하는 웹캠·라인 검사 시스템에 적합합니다.

3 충분한 보정 데이터 확보

실제 데이터와 비슷한 샘플을 300장 내외 준비할 수 있을 때.

4 정확도 재검증 가능

양자화 후 성능을 검증할 평가 파이프라인이 있을 때.

디바이스와 정밀도, 두 축을 함께 고려하기

개념

디바이스 축

- CPU / GPU / NPU / AUTO 중 목적에 맞게 선택.
- AUTO는 배포 환경이 다양할 때 특히 유용합니다.

정밀도 축

- FP32(정확) → FP16(균형) → INT8(속도) 순으로 트레이드오프.
- NNCF로 손쉽게 INT8 양자화를 적용할 수 있습니다.

직접 해보기

실습

TASK

동일한 검출 모델을 FP16/CPU, FP16/AUTO, INT8/CPU 세 조합으로 벤치마크하고 처리량·지연시간을 표로 정리해보세요.

TIP

다음 파트의 물류 자동 계수 프로젝트에서 바로 이 조합 선택 경험이 사용됩니다.

핵심 API 총정리

요약

항목	설명
`core.available_devices`	시스템에서 사용 가능한 디바이스 목록을 확인합니다.
`compile_model(model, "AUTO:...")`	우선순위를 지정해 자동으로 디바이스를 선택합니다.
`hints.performance_mode`	처리량/지연시간 우선 모드를 설정합니다.
`nncf.quantize(...)`	보정 데이터셋으로 모델을 INT8로 양자화합니다.
`ov.save_model(...)`	양자화된 모델을 IR 파일로 저장합니다.

참고사이트

실습

https://universe.roboflow.com/ - 데이터셋 , 학습등 일체
https://pixabay.com/videos - 학습에 필요한 참고 동영상들이 많이 있음
aihub.or.kr – 교통관련 동영상들
detrac-db.rit.albany.edu
kaggle

PART

2

물류 자동 계수 프로젝트

실시간 객체 검출과 추적으로 컨베이어 위 제품을 자동으로 세는 시스템을 만듭니다. Roboflow로 데이터를 라벨링·증강하고, 커스텀 모델을 학습·튜닝한 뒤 OpenVINO로 배포합니다.

물류 자동 계수 시스템이란

컨베이어 벨트나 입출고 게이트를 지나는 상자·제품을 카메라로 자동으로 세는 시스템입니다. 객체 검출 + 추적(Tracking) + 카운팅 로직의 조합으로 만들어집니다.

검출

프레임마다 상자의 위치를 찾아냅니다.

추적

같은 상자가 여러 프레임에서 같은 객체임을 인식합니다.

계수

기준선을 통과하거나 존을 벗어날 때 카운트를 올립니다.

시스템 구성 요소

구성요소	역할
카메라/영상	컨베이어 위를 촬영하는 고정 카메라 스트림.
검출 모델	YOLO26 기반 커스텀 검출 모델(상자 클래스).
추적기	프레임 간 동일 객체를 연결(ID 부여).
카운팅 로직	기준선/존 통과 이벤트를 카운트로 변환.

계수 로직: 라인 크로싱 vs 존 카운팅

개념

라인 크로싱(Line Crossing)

- 화면에 가상의 선을 긋고, 객체 중심이 선을 넘는 순간 카운트.
- 컨베이어처럼 한 방향 흐름에 적합합니다.

존 카운팅(Zone Counting)

- 관심 영역(ROI)에 들어오고 나가는 것을 기준으로 카운트.
- 여러 방향에서 진입하는 상황에 적합합니다.

검출 박스에서 중심점 구하기

코드

get_centroid.py

```
def get_centroid(xyxy):
    x1, y1, x2, y2 = xyxy
    return ((x1 + x2) / 2, (y1 + y2) / 2)
```

model.track()으로 다중 객체 추적하기

코드

track_demo.py

```
results = model.track(
    source=0, persist=True, device="cpu", tracker="bytetrack.yaml"
)
for r in results:
    for box in r.boxes:
        track_id = int(box.id) if box.id is not None else -1
```

NOTE persist=True는 프레임이 바뀌어도 같은 트래커 상태를 이어가게 합니다.

라인 크로싱 카운팅 로직

코드

line_counter.py

```
line_y = 300
prev_centroids = {}
count = 0

for box in r.boxes:
    tid = int(box.id)
    cx, cy = get_centroid(box.xyxy[0])
    prev_y = prev_centroids.get(tid, cy)
    if prev_y < line_y <= cy:
        count += 1
    prev_centroids[tid] = cy
```

카운트를 화면에 표시하기

코드

count_overlay.py

```
cv2.line(frame, (0, line_y), (frame.shape[1], line_y), (0,255,255), 2)
cv2.putText(frame, f"Count: {count}", (10, 40),
            cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0,0,255), 3)
```

중복 계수를 막는 방법

개념

문제 상황

- 트래킹 ID가 흔들리면 하나의 상자가 두 번 카운트될 수 있습니다.
- 겹친 상자가 잠깐 가려지면 새로운 ID가 부여되기도 합니다.

대응 전략

- 카운트된 ID는 집합(set)에 저장해 재카운트를 방지합니다.
- 트래커의 매칭 임계값과 프레임레이트를 함께 조정합니다.

계수 시스템에서 자주 겪는 문제

코드

항목	설명
카운트가 두 번씩 오름	ID 재사용을 막기 위해 카운트된 ID를 집합으로 관리하세요.
빠른 상자를 놓침	프레임레이트가 낮으면 선을 건너뛸 수 있습니다. 라인을 넓은 존으로 바꿔보세요.
겹친 상자 오검출	카메라 각도를 조정하거나 검출 신뢰도 기준을 높여보세요.

직접 해보기

실습

TASK

웹캠으로 움직이는 물체를 촬영하며 가상의 라인을 지나갈 때마다 카운트가 올라가는 데모를 만들어보세요.

TIP

손이나 컵처럼 손쉽게 반복 이동시킬 수 있는 물체로 먼저 테스트해보세요.

실전 배포 시 고려사항

1 카메라 고정

흔들리면 라인 위치가 틀어져 계수 오류가 늘어납니다.

2 조명 일정하게

명암 변화가 크면 검출 신뢰도가 불안정해집니다.

3 카운트 로그 저장

시간대별 카운트를 기록해두면 이상 발생 시 추적이 쉽습니다.

4 주기적 재검증

실제 개수와 시스템 카운트를 주기적으로 비교합니다.